Theme Report 2: Reason in Intelligent Embedded Systems

Vedant Jhawar
400507517
COMPENG 2DX3

Theme

Reason in embedded systems refers processing and making sense of observed data to react ti changes in the environment. While observation captures information, reasoning turns that into conclusions to shape system behavior. This whole process involves computation, logic, timing analysis, decision-making code that uses sensor data to perform more complex tasks. The quality of reasoning directly determines whether the system responds intelligently to its environment or behaves inappropriately despite having accurate observational data.

A good example of reasoning would be temperature sensing in a smart thermostat. The observation phase provides temperature readings from multiple sensors throughout a building. Reasoning uses those readings, factors in sensor placement and biases, and calculates averages. Then, it compares these against preset values and determines whether to activate heating, cooling, or maintain current status. Outlier data points must also be handled appropriately, perhaps an alert should be sent rather than increasing/decreasing temperature.

Reasoning involves different kinds of operations. For example, arithmetic operations for calculating values like duty cycles and motor speeds, logical operations for implementing state machines, and timing calculations for generating waveforms. These activities connect input signals to useful output actions, making reasoning a very important stage for an embedded system.

Background

Reasoning builds upon the observation phase mentioned earlier. Without observation, there is nothing to reason about. Without reasoning, there is nothing to draw from observations and no decisions can be made. The reason phase is where embedded systems recognize patterns, predict outcomes, and execute decisions.

Consider an autonomous drone. The observation phase provides continuous data streams: GPS coordinates for position, accelerometer and gyroscope readings for orientation, downward-facing cameras for position hold, and time-of-flight sensors for obstacle detection. Alone, each stream is just numbers meaningless without context.

The drone's reasoning systems must fuse these observations into coherent understanding. The flight controller combines accelerometer and gyroscope data through sensor fusion algorithms like complementary filters, accounting for each sensor's strengths. Gyroscopes respond quickly but drift over time, while accelerometers are stable long-term but noisy during motion. The resulting attitude estimate informs motor mixing calculations that keep the drone level.

For navigation, the drone reasons about position relative to waypoints. GPS provides absolute position but with potential errors and dropouts. The reasoning system may combine GPS with optical flow from the downward camera to maintain position when GPS is unreliable. When the drone approaches a tree, time-of-flight sensor readings trigger obstacle avoidance reasoning: should it stop, climb, or go around? This decision considers current speed, available escape paths, battery remaining, and mission priorities.

A failure in reasoning can be catastrophic. Misinterpreting wind as pilot input might cause overcorrection and flipping. Failing to recognize distance sensor readings due to sunlight interference could lead to collision with actual obstacles. Even with perfect sensors, poor reasoning means the drone cannot understand its environment well enough to act appropriately.

Labs 4 through 6 progressively develop reasoning capabilities. Lab 4 introduces PWM and stepper motor control, requiring calculation of timing parameters and generation of precise control sequences. Lab 5 extends reasoning to user interface decoding, where raw keypad scan codes are interpreted into meaningful button values. Lab 6 integrates reasoning across multiple subsystems, combining motor control and sensor interpretation into a coherent system that can execute coordinated actions.

Theme Exemplars

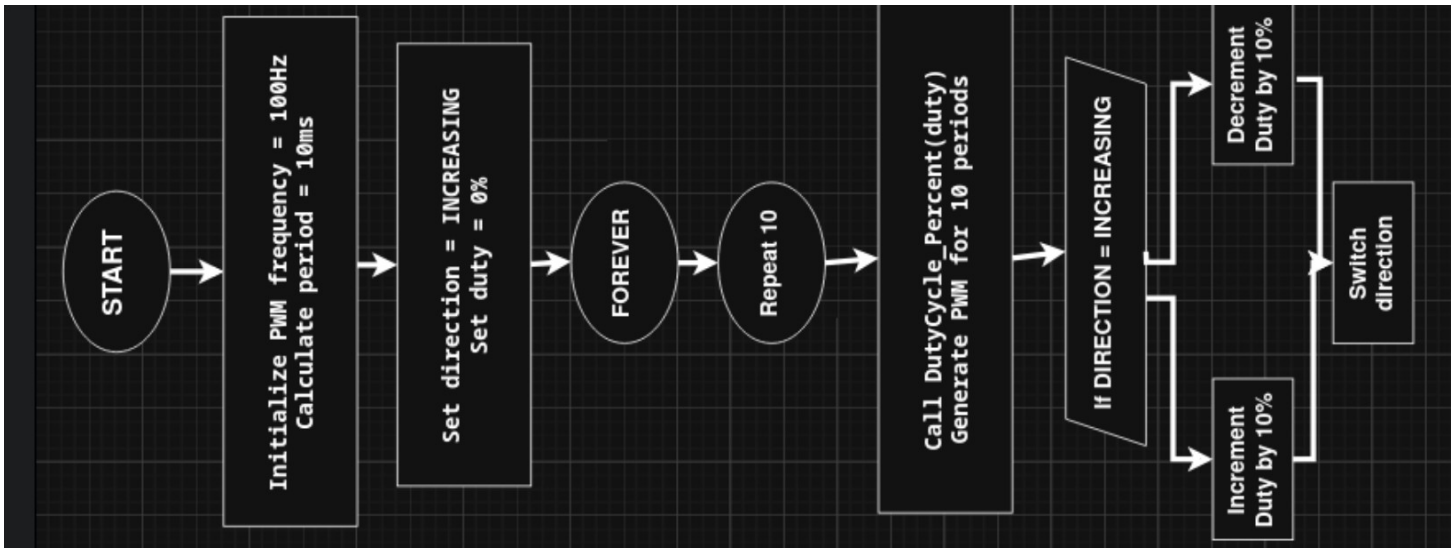
Exemplar 1: PWM Generation and Control (Lab 4)

Method

Lab 4 required implementing a variable duty cycle PWM signal to control LED brightness and ultimately stepper motor motion. The reasoning challenge lay in calculating timing parameters, managing state transitions, and generating precise control sequences.

For the IntensitySteps function, I created a systematic approach to duty cycle variation. The function called DutyCycle_Percent with an unsigned integer input representing the desired duty cycle out of 255. The reasoning involved converting this percentage value into actual timing delays: for a 100 Hz square wave with period of 10 ms, a 10% duty cycle required 1 ms high time and 9 ms low time. The code calculated these values mathematically based on the input parameter.

The intensity stepping pattern required additional reasoning about state sequencing. The program needed to increment duty cycle from 0% to 100% in 10% steps, then decrement back down, repeating each step ten times before moving to the next. This required careful loop control and boundary checking to ensure smooth transitions without overflow or underflow errors. The flowchart below illustrates the reasoning structure:



The DutyCycle_Percent function performed the core reasoning: accepting a percentage value (0-100) scaled to an 8-bit input (0-255), calculating high time and low time, and generating the appropriate waveform through precise delay loops.

For the stepper motor control, reasoning extended to sequence generation and timing analysis. The motor required four control lines driven in a specific pattern (0b0011, 0b0110, 0b1100, 0b1001) to achieve full-step rotation. The reasoning challenge was determining the minimum delay between steps

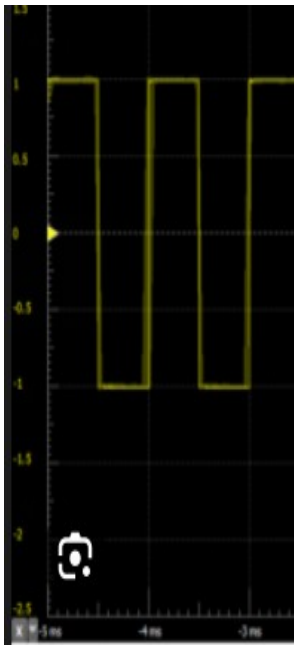
empirically while ensuring reliable rotation. I implemented a variable delay parameter and systematically reduced it while observing motor behavior to find the threshold where stalling occurred.

Validation Results

Validation of PWM generation used the Analog Discovery 3 to verify both frequency and duty cycle accuracy. At 10% duty cycle, the scope showed exactly 1 ms high time within a 10 ms period. As duty cycle stepped through values, the waveform width changed proportionally, confirming correct mathematical conversion from percentage to timing. The LED brightness visibly changed in smooth steps, providing visual confirmation of the reasoning process.

For the stepper motor, empirical testing revealed a minimum delay of 1.8 ms between steps for reliable rotation. Below this value, the motor would stall or vibrate without turning. At the calculated minimum, the motor completed two full rotations clockwise followed by one counter-clockwise rotation exactly as programmed.

The AD3 capture below shows the PWM waveform at 50% duty cycle, demonstrating precise timing generation through calculated delay values:



Exemplar 2: Keypad Decoding and Interpretation (Lab 5)

Method

Lab 5 presented a sophisticated reasoning challenge: converting raw electrical signals from a 4x4 keypad into meaningful button values that could drive system behavior. The keypad used a matrix scanning technique where output pins drive columns and input pins read rows. The reasoning occurred in multiple stages, each transforming data into increasingly abstract representations.

The first stage involved scanning the keypad to generate an 8-bit code representing which button was pressed. The scanning algorithm systematically drove each column low while reading all four row inputs. When a button was pressed, it connected its column to its row, pulling that row input low. The reasoning challenge was to interpret these electrical states correctly, accounting for the fact that multiple columns might show activity if multiple buttons were pressed. The scanning code structure implemented the following reasoning:

```c
uint8_t Keypad_Get8bitCode(void){
  static const uint8_t row_patterns[4] = {
    0x0E, // 1110  (PE0 low)
    0x0D, // 1101  (PE1 low)
    0x0B, // 1011  (PE2 low)
    0x07  // 0111  (PE3 low)
  };

  for(int r=0; r<4; r++){
    GPIO_PORTE_DATA_R = row_patterns[r];
    settle_delay();

    uint8_t cols = GPIO_PORTM_DATA_R & 0x0F; // PM nibble
    if(cols != 0x0F){                        // any column went low => key pressed
      uint8_t rows = GPIO_PORTE_DATA_R & 0x0F;
      return (cols << 4) | rows;             // PM[3:0] PE[3:0]
    }
  }
  return 0xFF;
}
```

This raw 8-bit code represented the physical electrical state but had no inherent meaning. The second reasoning stage performed decoding: mapping each unique 8-bit code to a logical 4-bit binary value representing the button number (0-15). This required building a lookup table or implementing a matching algorithm that could recognize patterns regardless of minor electrical variations.

The decoding function implemented the following reasoning:

```c
uint8_t decode_keypad(uint8_t scan_code) {
    switch(scan_code) {
        case 0xEE: return 0x0;  // Button 0
        case 0xDE: return 0x1;  // Button 1
        case 0xBE: return 0x2;  // Button 2
        case 0x7E: return 0x3;  // Button 3
        case 0xED: return 0x4;  // Button 4
        // ... additional cases for all 16 buttons
        default:   return 0xFF;  // Invalid
    }
}
```

The final reasoning stage translated the 4-bit binary value into physical LED outputs according to a group-specific bit ordering. This required mapping bits to specific LEDs based on the sum of student numbers, implementing a configurable output mapping that preserved the logical meaning while adapting to physical wiring constraints.

Validation Results

Validation proceeded through systematic testing. Each button was pressed individually while monitoring the scan code in the Keil debugger. The observed codes matched expected patterns: pressing button in row 0, column 0 produced 0xEE; button in row 0, column 1 produced 0xDE, and so on. All 16 buttons produced unique, repeatable codes, confirming that the scanning algorithm correctly interpreted the matrix connections.

The decoding stage was validated by pressing each button and verifying that the returned 4-bit value matched the expected button number. Button 0 returned 0x0, button 1 returned 0x1, through button 15 returning 0xF. This confirmed that the mapping from electrical state to logical value was correct.

Finally, LED output validation confirmed that the 4-bit value was displayed correctly on the onboard LEDs according to the assigned ordering. For my group (sum LSD even), bit 3 mapped to LED D4, bit 2 to LED D3, bit 1 to LED D2, and bit 0 to LED D1. Pressing button 5 (binary 0101) lit LEDs D3 and D1 while leaving D4 and D2 off, exactly matching expectations.

The table below summarizes the mapping validation:

Button	Binary Value	Expected LEDs	Observed LEDs
0	0b0	None	None
1	0b001	D1	D1
2	0b010	D2	D2
3	0b011	D2, D1	D2, D1
4	0b100	D3	D3

5	0b101	D3, D1	D3, D1

4. Debugging Exemplar

Functional Debugging Using Watch Variables and Memory Inspection

During the Reason theme labs, debugging shifted from hardware-focused verification to software-focused reasoning validation. The most effective technique I employed was functional debugging using watch variables and memory inspection, which allowed me to observe the internal state of my reasoning algorithms without disrupting their execution.

Context: Keypad Decoding Logic Error

In Lab 5, during initial testing of the keypad decoding function, I encountered a puzzling problem: pressing button 7 produced the correct LED pattern for button 3, while button 3 produced no output at all. The physical scanning appeared to work because the LEDs responded consistently but the mapping was clearly wrong. This could have been a hardware issue (incorrect wiring causing swapped columns), a scanning algorithm error (misidentifying which column was active), or a decoding table mistake.

Debugging Approach

Rather than guessing, I used Keil's watch window to inspect the reasoning process at each stage. I added three key variables to the watch list:

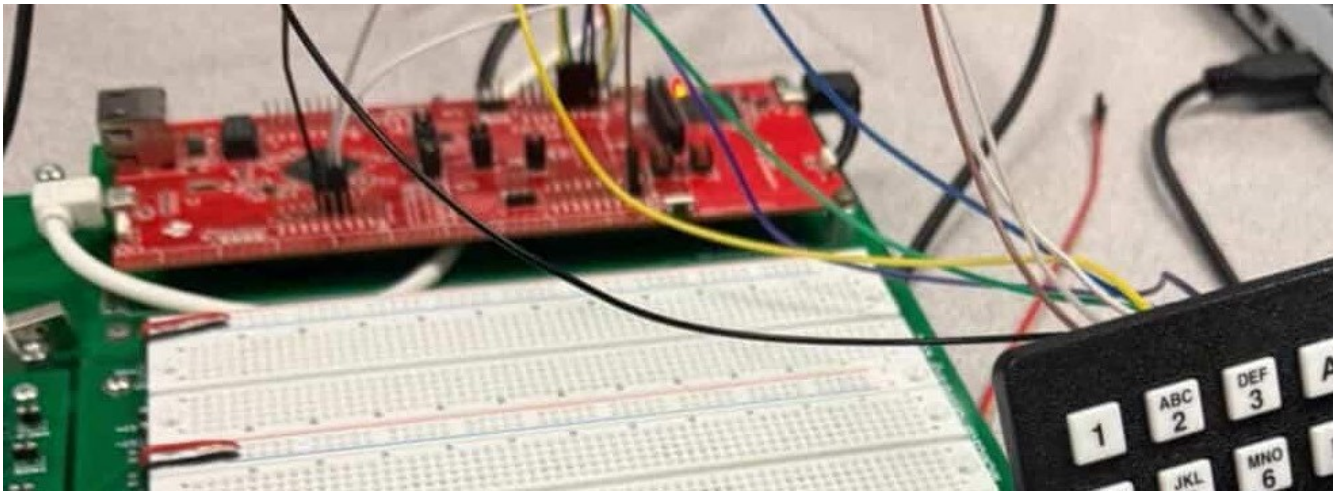
1. `scan_code`: The raw 8-bit value from the scanning function
2. `decoded_value`: The 4-bit result from the decoding function
3. `led_output`: The final value written to the LED port

With breakpoints set after each transformation, I could step through the reasoning chain and observe exactly where the error occurred.

The debug session revealed:

- Pressing button 7 produced `scan_code = 0x7B`
- The decoding function mapped `0x7B` to `decoded_value = 0x3`
- `led_output` correctly displayed binary 0011 on the LEDs

This showed that the scanning was correct (button 7 produced a unique code) and the LED output was correct (0011 displayed properly), but the decoding table had an incorrect entry. The error was purely in the reasoning logic. My mapping table associated `0x7B` with button 3 instead of button 7.



Resolution

I corrected the decoding table entry and verified the fix by observing the watch window update with the proper values. The image below shows the Keil debugger with watch variables displaying the correct data flow:

This debugging approach proved invaluable because it isolated the reasoning layer from both hardware and output layers. Without inspecting intermediate values, I might have wasted time checking wiring, testing continuity, or modifying LED output code. All irrelevant to the actual problem.

Application to PWM Debugging

In Lab 4, a similar functional debugging approach helped verify PWM timing calculations. I added watch variables for `high_time_us`, `low_time_us`, and `current_duty_cycle`, then used the oscilloscope to cross-reference calculated values against actual waveforms. When the duty cycle stepping pattern seemed irregular, the watch variables revealed that my boundary condition logic was skipping 100% duty cycle due to an off-by-one error in the increment loop.

The combination of software watch variables and hardware measurement tools provided comprehensive visibility into both the reasoning process and its physical manifestation, enabling timely identification and correction of logical errors.

5. Synthesis

The exemplars illustrate how reasoning transforms raw data into meaningful action. Both demonstrate that reasoning is not a single operation but a chain of transformations, each adding abstraction and meaning to information flowing through the system.

In the PWM exemplar, reasoning begins with abstract requirements: produce a 100 Hz waveform with duty cycle stepping from 0% to 100%. This translates into discrete steps: calculate period from frequency, convert percentage to timing values, generate delays, and manage state transitions. The reasoning is sequential, converting specifications into precise timing control. Oscilloscope validation confirms the translation from abstract percentages to physical waveforms.

The keypad exemplar demonstrates interpretative decoding. Raw electrical signals contain no inherent meaning. The scanning algorithm imposes structure by enforcing an ordered scan pattern. The decoding function interprets this structured data by matching patterns against a known mapping, producing logical values corresponding to user intent. Finally, LED output mapping translates logical values back into physical signals.

Both exemplars share characteristics defining embedded reasoning: abstraction (voltages → bits → codes → values → actions actions), determinism (predictable, repeatable outputs), timing awareness (operations completing within constraints), and error handling (accounting for invalid inputs).

These elements create intelligent behavior. A system that simply reads buttons and lights LEDs is trivial. A system that interprets button presses in context, generates complex waveforms based on calculated parameters, and coordinates multiple actuators demonstrates genuine reasoning. Without reasoning, observation is meaningless and action impossible. The reason phase provides the intelligence distinguishing a responsive system from a reactive one.

6. Reflection

a) Understanding of the Reason Theme

Before these labs, I understood reasoning as simply "making decisions based on input." The exemplars expanded this understanding dramatically. Reasoning is not a single decision point but an entire pipeline of transformations, each adding value to raw data.

The PWM lab taught me that reasoning includes precise mathematical calculation. Generating a specific waveform requires understanding relationships between time, frequency, and percentage. And also how to implement them reliably on resource-constrained hardware. Figuring out minimum stepper motor delay showed that reasoning sometimes requires experimental data collection, not just theoretical calculation.

The keypad lab revealed reasoning as interpretation and abstraction. The same electrical signal, a pin reading logic low, could mean "button 1" or "button 7" depending entirely on the scanning sequence.

Most importantly, I learned that reasoning must be verified at every stage. The debugging exemplar showed errors can hide in any transformation layer, and inspecting values is essential for isolating problems. This has made me more systematic in both design and debugging.

b) Understanding of Intelligent Systems and Microprocessors

These labs deepened my understanding of how microprocessors enable intelligent behavior. The microprocessor is not just executing instructions; it continuously transforms data streams through multiple abstraction layers. Each peripheral like timers, GPIO, I2C provide capabilities that reasoning software leverages to interact with the physical world.

The progression from Lab 4 to Lab 6 showed how reasoning complexity scales. Lab 4's reasoning was contained within single functions performing calculations. Lab 5's reasoning involved multiple functions requiring coordination between scanning, decoding, and output stages. Lab 6 integrated motor control, sensor reading, and user interface into a unified system where reasoning coordinated multiple subsystems simultaneously.

I also gained appreciation for embedded reasoning constraints. Limited memory demands efficient lookup tables. Real-time requirements demand calculations complete within strict timing windows. Power constraints require minimizing computational overhead. These constraints force careful design and optimization, distinguishing embedded reasoning from general-purpose computing.

The debugging techniques, watch variables, memory inspection, and AD3 verification, gave visibility into the microprocessor's internal reasoning processes. Seeing variables update in real-time as the system responded to inputs made the abstract concept of "reasoning" concrete and observable. This visibility will be essential as I tackle more complex embedded systems.

7. References

[1] J. W. Valvano, *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers*, 5th ed. CreateSpace Independent Publishing Platform, 2019.

[2] Texas Instruments, "MSP432E401Y Microcontroller Datasheet," 2023.

[3] Texas Instruments, "MSP432E4 SimpleLink Microcontrollers Technical Reference Manual," 2023.

[4] Keil, "MDK Debugger User's Guide," ARM Limited, 2024.

[5] Velleman, "VMA401 Stepper Motor with Driver User Guide," 2023.

[6] Grayhill, "96BB2-006-F 4x4 Keypad Datasheet," 2023.

[7] Laboratory Manual — PWM and Stepper Motor Control, Finite State Machines, and Keypad Interfacing, COMPENG 2DX3, McMaster University, 2026.